Case Study: Employee–Department Management System – Phase 2

1. Background

Your organization currently has a Spring Boot application that manages:

- Departments
- Employees
- Employee-to-department relationships
- PostgreSQL database integration
- Global exception handling
- JWT-based authentication
- Role-based access using `ADMIN` and `USER`
- Method-level security using `@PreAuthorize`

The organization now wants to enhance the application into an enterprise-level **Workforce Management System**.

The new version must support department managers, employee transfers, employee status management, audit information, advanced searching, pagination, reporting and stronger authorization rules.

---

2. Existing Data Model

Department

The current `Department` entity contains fields such as:

- Department ID
- Department code
- Department name
- Department description
- Location

A department can have multiple employees.

Employee

The current `Employee` entity contains fields such as:

- Employee ID
- Employee code

- First name
- Last name
- Email
- Salary
- Joining date
- Department

Each employee belongs to one department.

Application User

The application user contains:

- User ID
- Username
- Password
- Role
- Enabled status

Existing roles:

- `ADMIN`
- `USER`

---

3. Business Requirement

The organization wants to introduce a new role called:

```
MANAGER
```

A manager should be associated with exactly one department.

The application must enforce the following responsibilities.

ADMIN

An administrator can:

- Create departments
- Update departments
- Delete departments
- Create employees
- Update employees

- Delete employees
- Transfer employees
- Create application users
- Assign roles
- Assign managers to departments
- View all employees and departments
- Activate or deactivate employees
- View audit information

MANAGER

A manager can:

- View their own department
- View employees belonging to their department
- Update limited employee information
- Approve or reject transfer requests
- Activate or deactivate employees in their department
- View department statistics
- Search employees within their department

A manager cannot:

- Delete departments
- Create administrators
- View employees from another department
- Change an employee’s salary
- Assign application roles
- Delete an employee permanently

USER

A normal user can:

- View their own profile
- View their own department
- Submit a department transfer request
- Update limited personal information
- Change their password

A normal user cannot:

- View salary information of other employees
- View all employees
- Create or delete departments
- Approve transfer requests
- Assign roles

4. New Functional Requirements

Requirement 1: Employee Status

Add an employee status field.

Supported values:

```
ACTIVE
INACTIVE
ON_LEAVE
RESIGNED
TERMINATED
```

Validation Rules

- A newly created employee must have `ACTIVE` status.
- A resigned employee cannot be transferred.
- A terminated employee cannot be updated by a manager.
- An inactive employee cannot log in.
- Once an employee is terminated, only an administrator can reactivate the employee.
- An employee with `RESIGNED` status cannot be changed back to `ACTIVE`.

Requirement 2: Department Manager

Each department may have one manager.

A manager must also be an employee belonging to the same department.

Validation Rules

- An employee cannot manage more than one department.
- An employee must have `MANAGER` role before being assigned as a department manager.
- A department manager must belong to the department they manage.
- A manager cannot be transferred while assigned as the department manager.
- A department cannot have two active managers.
- Removing a manager should not delete the employee.

Requirement 3: Employee Transfer Request

Employees should no longer be transferred directly by normal users.

Create a transfer-request feature.

A transfer request should contain:

- Transfer request ID
- Employee
- Current department
- Requested department
- Reason
- Request date
- Status
- Reviewed by
- Reviewed date
- Review comments

Supported transfer statuses:

```
PENDING
APPROVED
REJECTED
CANCELLED
```

Business Rules

- An employee cannot request a transfer to their current department.
- Only active employees can submit transfer requests.
- An employee can have only one pending transfer request.
- The transfer reason must contain between 20 and 500 characters.
- A manager can review only transfer requests related to their department.
- An employee cannot approve their own transfer.
- Once approved, the employee’s department must be updated automatically.
- An approved transfer request cannot be cancelled.
- A rejected request must contain review comments.
- Only the employee who created the request can cancel it.
- Only pending requests can be cancelled.
- Managers cannot approve the transfer of a department manager.
- Administrators can override manager decisions.

Requirement 4: Salary Security

Salary is confidential information.

Authorization Rules

- `ADMIN` can view and update every employee's salary.
- `MANAGER` can view employee salary only within their department.
- `MANAGER` cannot modify salary.
- `USER` can view only their own salary.
- Salary must not appear in unauthorized API responses.
- Salary information must not be exposed simply by hiding the field in the frontend.

Participants should create separate response DTOs for different authorization levels.

Requirement 5: Audit Information

Add audit fields to important entities.

Suggested fields:

```
createdAt
createdBy
updatedAt
updatedBy
```

Enable Spring Data JPA auditing.

Audit Requirements

- Audit fields should be generated automatically.
- `createdAt` should not change after creation.
- `updatedAt` should change whenever the entity is updated.
- `createdBy` and `updatedBy` should store the logged-in username.
- Audit information should be visible only to administrators.
- Passwords and JWT tokens must never be stored in audit records.

Requirement 6: Soft Delete

Employees should not be permanently deleted from the database.

Add a soft-delete mechanism.

Suggested field:

```
deleted
```

Rules

- Delete operations should mark the employee as deleted.
- Deleted employees should not appear in normal employee searches.
- Only administrators can view deleted employees.
- A deleted employee cannot log in.
- Email addresses of deleted employees should remain reserved.
- An administrator may restore a deleted employee.
- Restoring an employee should not automatically reactivate the application account.

Requirement 7: Search, Sorting and Pagination

Create an employee-search API.

Search criteria may include:

- Employee code
- First name
- Last name
- Email
- Department
- Employee status
- Joining date range
- Minimum salary
- Maximum salary

The search API should support:

- Page number
- Page size
- Sorting field
- Sorting direction
- Multiple optional search filters

Example:

```
GET /api/employees/search?
page=0&size=10&departmentId=1&status=ACTIVE&sort=firstName,asc
```

Rules

- Maximum page size should be 100.
- Invalid sorting fields must generate a validation error.
- Managers should receive only employees from their department.
- Users should not have access to the general search API.
- Deleted employees should be excluded by default.

Requirement 8: Department Statistics

Create a department-statistics API.

The response should include:

- Department ID
- Department name
- Total employees
- Active employees
- Inactive employees
- Employees on leave
- Average salary
- Minimum salary
- Maximum salary
- Number of pending transfer requests

Authorization

- Administrators can view statistics for all departments.
- Managers can view statistics only for their department.
- Normal users cannot access salary statistics.

---

5. Suggested New APIs

Authentication APIs

```
POST /api/auth/login
POST /api/auth/change-password
GET  /api/auth/me
```

Department APIs

```
POST   /api/departments
GET    /api/departments
GET    /api/departments/{id}
PUT    /api/departments/{id}
DELETE /api/departments/{id}
PUT    /api/departments/{departmentId}/manager/{employeeId}
DELETE /api/departments/{departmentId}/manager
GET    /api/departments/{departmentId}/statistics
```

Employee APIs

```
POST   /api/employees
GET    /api/employees
GET    /api/employees/{id}
PUT    /api/employees/{id}
DELETE /api/employees/{id}
PATCH  /api/employees/{id}/status
PATCH  /api/employees/{id}/salary
POST   /api/employees/{id}/restore
GET    /api/employees/search
GET    /api/employees/me
PUT    /api/employees/me
```

Transfer Request APIs

```
POST  /api/transfer-requests
GET   /api/transfer-requests/my-requests
GET   /api/transfer-requests/pending
GET   /api/transfer-requests/{id}
PATCH /api/transfer-requests/{id}/approve
PATCH /api/transfer-requests/{id}/reject
PATCH /api/transfer-requests/{id}/cancel
```

User Management APIs

```
POST  /api/users
GET   /api/users
GET   /api/users/{id}
```

```
PATCH /api/users/{id}/role
PATCH /api/users/{id}/status
```

---

6. Suggested `@PreAuthorize` Requirements

Participants should implement method-level authorization.

Examples of expected rules:

```
@PreAuthorize("hasRole('ADMIN')")
```

Use for:

- Department creation
- Department deletion
- Salary modification
- User creation
- Role assignment
- Employee restoration

```
@PreAuthorize("hasAnyRole('ADMIN', 'MANAGER')")
```

Use for:

- Employee search
- Department statistics
- Viewing transfer requests

Participants should also implement ownership-based authorization.

Examples:

```
@PreAuthorize("hasRole('ADMIN') or @employeeSecurity.isOwner(#employeeId)")
```

```
@PreAuthorize("hasRole('ADMIN') or
@departmentSecurity.isManagerOfDepartment(#departmentId)")
```

```
@PreAuthorize("hasRole('ADMIN') or @transferSecurity.canReview(#requestId)")
```

The authorization logic should not depend only on roles. It should also check data ownership and department membership.

---

7. Validation Requirements

Participants must implement the following validations.

Department Validation

- Department code is mandatory.
- Department code must be unique.
- Department code must contain 2 to 10 uppercase characters.
- Department name is mandatory.
- Department name must contain between 3 and 100 characters.
- A department with active employees cannot be deleted.
- A department cannot be deleted when pending transfer requests exist.

Employee Validation

- Employee code must be unique.
- Email must be unique.
- Email must have a valid format.
- First name and last name are mandatory.
- Salary must be greater than zero.
- Joining date cannot be in the future.
- Department must exist.
- Employee status must be valid.
- An employee cannot be their own manager.
- A manager must belong to the assigned department.

Password Validation

The password must:

- Contain at least eight characters
- Contain one uppercase letter
- Contain one lowercase letter
- Contain one digit
- Contain one special character
- Not contain the username
- Not match the previous password

---

8. Required Exception Classes

Participants should create meaningful business exceptions.

Suggested exceptions:

```
ResourceNotFoundException
DuplicateResourceException
InvalidEmployeeStatusException
InvalidTransferRequestException
PendingTransferRequestExistsException
DepartmentManagerConflictException
UnauthorizedDepartmentAccessException
EmployeeNotActiveException
InvalidPasswordException
BusinessValidationException
```

The global exception handler should produce a consistent response.

Suggested error response:

```
{
  "timestamp": "2026-07-28T14:30:00",
  "status": 400,
  "error": "Bad Request",
  "message": "Employee already has a pending transfer request",
  "path": "/api/transfer-requests",
  "validationErrors": []
}
```

---

9. Database Requirements

Participants should update the PostgreSQL design.

Suggested tables:

```
departments
employees
application_users
roles
```

```
user_roles
transfer_requests
```

Important constraints:

- Department code must be unique.
- Employee code must be unique.
- Employee email must be unique.
- Username must be unique.
- Employee department ID must be a foreign key.
- Department manager ID must reference an employee.
- Transfer request employee ID must reference an employee.
- Current department ID must reference a department.
- Requested department ID must reference a department.
- Reviewer ID must reference an application user.
- Current department and requested department must be different.

Participants should determine which rules belong in:

- Java validation
- Service-layer validation
- Database constraints
- Security configuration

---

10. Participant Implementation Tasks

Task 1: Add the Manager Role

Add `MANAGER` to the role model and update the JWT so that manager authority is included in the token.

Task 2: Map Users to Employees

Associate an application user account with an employee.

Ensure that one application user cannot be linked to multiple employees.

Task 3: Assign Department Managers

Implement manager assignment with all required validation rules.

Task 4: Implement Transfer Requests

Create the entity, repository, service, controller, DTOs and exception handling for employee transfers.

Task 5: Implement Ownership-Based Security

Create reusable Spring beans for security expressions.

Suggested beans:

```
employeeSecurity
departmentSecurity
transferSecurity
```

Task 6: Protect Salary Information

Create separate DTOs so confidential salary data is returned only to authorized users.

Task 7: Implement Pagination and Search

Use Spring Data JPA specifications, criteria queries or repository query methods.

Task 8: Implement JPA Auditing

Capture the authenticated username in `createdBy` and `updatedBy`.

Task 9: Implement Soft Delete

Ensure all normal repository queries exclude deleted employees.

Task 10: Update Postman Collection

Add requests for:

- Admin login
- Manager login
- User login
- Department-manager assignment
- Employee transfer submission
- Transfer approval
- Transfer rejection
- Unauthorized salary access
- Manager accessing another department
- Employee soft deletion
- Employee restoration

---

11. Participant Discussion Questions

Spring Data JPA Questions

1. Why is `Employee` the owning side of the employee-department relationship?
2. Where should `mappedBy` be used in the relationship?
3. What can happen if `CascadeType.ALL` is applied from `Department` to `Employee`?
4. Should deleting a department automatically delete its employees? Explain.
5. What is the difference between lazy loading and eager loading?
6. Why can returning JPA entities directly from controllers cause recursive JSON serialization?
7. How can an `N+1` query problem occur while loading departments and employees?
8. When should `@EntityGraph` or a fetch join be used?
9. How would you implement optional employee-search filters?
10. What is the difference between `findById()` and `getReferenceById()`?
11. Should the transfer request store only the current employee department relationship, or should it also preserve the original department ID?
12. Why should historical transfer information not depend entirely on the employee's current department?

PostgreSQL Questions

1. Which table contains the foreign key in the employee-department relationship?
2. What should happen when an attempt is made to delete a department containing employees?
3. Which validations should be implemented using database constraints?
4. Can a database `CHECK` constraint ensure that a manager belongs to the same department? Explain.
5. Why should employee email remain unique even after soft deletion?

6. What indexes should be created for employee search?
7. Should salary use `double`, `float` or `BigDecimal`? Explain.
8. How would you prevent two pending transfer requests for the same employee at the database level?
9. What is the purpose of a database transaction during transfer approval?
10. What could happen if two managers approve the same transfer request simultaneously?

Spring Security Questions

1. What is the difference between authentication and authorization?
2. What is stored inside the JWT?
3. Why should passwords never be stored inside JWTs?
4. What is the purpose of `@EnableMethodSecurity`?
5. What is the difference between `hasRole("ADMIN")` and `hasAuthority("ROLE_ADMIN")`?
6. Why is URL-based authorization alone insufficient for this application?
7. How can a manager be prevented from accessing another department?
8. Why should salary security be implemented in the backend rather than only in the frontend?
9. What HTTP status code should be returned when the JWT is missing?
10. What HTTP status code should be returned when the user is authenticated but lacks permission?
11. What is the difference between `401 Unauthorized` and `403 Forbidden`?
12. Should user roles always be loaded from the JWT, or should they be checked against the database for every request?
13. What happens to existing JWTs after an administrator disables a user?
14. How can immediate token revocation be implemented?

Validation and Exception-Handling Questions

1. What is the difference between DTO validation and business validation?
2. Where should the validation “manager must belong to the same department” be implemented?
3. Why should repository exceptions not be returned directly to clients?
4. Should duplicate email errors return HTTP `400` or `409`?
5. What information should not be included in an API error response?
6. Why should stack traces not be returned in production?
7. How should multiple field-validation errors be represented?
8. Which exception should be raised when an employee has already submitted a pending transfer request?
9. Should transfer rejection comments be validated in the DTO or service layer?

Transaction Questions

1. Why should transfer approval use `@Transactional`?
2. Which operations should happen inside the same transfer transaction?
3. What should happen if the employee department is updated but updating the transfer-request status fails?
4. What is transaction rollback?
5. What is the default rollback behavior for checked and unchecked exceptions?
6. How can optimistic locking prevent simultaneous updates?
7. Where could `@Version` be added in this application?
8. What happens when two administrators update the same employee at the same time?

---

12. Scenario-Based Questions

Scenario 1: Invalid Department Manager

An administrator tries to assign employee `EMP1005` as the manager of the Finance department.

However, `EMP1005` currently belongs to the IT department.

Questions

1. Should the assignment be allowed?
2. Which layer should validate the department membership?
3. Which exception should be raised?
4. Which HTTP status code should be returned?
5. Should the application automatically transfer the employee to Finance?

Scenario 2: Unauthorized Salary Update

A manager sends the following request:

```
PATCH /api/employees/25/salary
```

The manager belongs to the same department as employee 25.

Questions

1. Should the manager be allowed to update the salary?
2. Should the request fail at the URL security level or method security level?
3. Which HTTP status code should be returned?
4. How should the authorization rule be written?
5. Should the manager be able to view the salary?

Scenario 3: Duplicate Pending Transfer

An employee already has a pending transfer request from IT to Finance.

The employee submits another transfer request from IT to Sales.

Questions

1. Should the second request be accepted?

2. How can this rule be checked using Spring Data JPA?
3. How can concurrency create duplicate pending requests?
4. Can a database index help prevent this problem?
5. What error response should be returned?

Scenario 4: Manager Transfer

The manager of the Finance department submits a request to transfer to the IT department.

Questions

1. Should the transfer request be accepted?
2. Must another manager be assigned first?
3. Who can approve the request?
4. What should happen to the department-manager relationship after approval?
5. Which operations should be part of the same transaction?

Scenario 5: Soft-Deleted Employee Login

An administrator soft-deletes an employee, but the employee still has a valid JWT.

Questions

1. Should the existing JWT continue to work?
2. How can the application check the employee status during authentication?
3. Is a stateless JWT sufficient for immediate revocation?
4. What token-revocation mechanisms can be introduced?
5. Should the user account and employee record have separate enabled flags?

Scenario 6: Simultaneous Transfer Approval

Two administrators open the same pending transfer request.

Both click Approve at nearly the same time.

Questions

1. What concurrency issue may occur?
2. How can optimistic locking solve it?
3. Where should `@Version` be added?
4. What response should be returned to the second administrator?
5. Should the employee be transferred twice?

Scenario 7: Deleting a Department

An administrator attempts to delete a department that has:

- Five active employees
- One inactive employee
- Two pending transfer requests

Questions

1. Should the department be deleted?
2. Which conditions should block deletion?
3. Should employees be automatically moved to another department?
4. Should a department support soft deletion?
5. Which HTTP status code should be returned?

---

13. API Testing Questions

1. How will you test an API without providing a JWT?
2. How will you test a normal user accessing an admin endpoint?
3. How will you verify that managers cannot access another department?
4. How will you test expired JWT handling?
5. How will you test invalid login credentials?
6. How will you verify that passwords are encrypted in PostgreSQL?
7. How will you test duplicate email validation?
8. How will you verify transaction rollback?
9. How will you test pagination boundaries?
10. How will you test an invalid employee status?
11. How will you test a rejected transfer without review comments?
12. How will you verify that a soft-deleted employee does not appear in searches?
13. How will you verify that salary is missing from an unauthorized response?

14. How will you store admin, manager and user tokens separately in Postman?

15. How will you create Postman tests for expected `401`, `403`, `404` and `409` responses?

---

14. Expected Postman Workflow

Participants should execute the following workflow:

1. Login as administrator.
2. Create an IT department.
3. Create a Finance department.
4. Create an employee in the IT department.
5. Create a manager user.
6. Associate the manager user with the employee.
7. Assign the employee as the IT department manager.
8. Login as the manager.
9. Retrieve IT department employees.
10. Attempt to retrieve Finance employees.
11. Verify that the request returns `403 Forbidden`.
12. Create a normal employee user.
13. Login as the normal user.
14. Submit a transfer request from IT to Finance.
15. Login as the manager.
16. Approve or reject the transfer request.
17. Verify the employee's department after approval.
18. Attempt to update salary as the manager.
19. Verify that the request is rejected.
20. Login as administrator and update the salary.
21. Soft-delete the employee.
22. Verify that the deleted employee cannot log in.
23. Restore the employee.
24. Verify that restoration does not automatically enable login.

---

15. Deliverables Expected from Participants

Participants must submit:

- Updated entity classes
- DTO classes
- Repository interfaces
- Service interfaces
- Service implementations
- REST controllers

- Security configuration
- JWT implementation
- Custom security-expression beans
- Global exception handler
- PostgreSQL schema
- `application.yaml`
- Unit tests
- Integration tests
- Updated Postman collection
- ER diagram
- API endpoint documentation
- Role-permission matrix
- README with setup instructions

---

16. Evaluation Criteria

Area	Weight
Entity relationships and database design	15%
Business validations	15%
Spring Security and authorization	20%
Service-layer design	10%
Exception handling	10%
Transaction management	10%
API design and DTO usage	10%
Testing and Postman collection	10%

---

17. Bonus Challenges

Participants who complete the main requirements can attempt the following:

Bonus 1: Refresh Tokens

Implement access tokens and refresh tokens.

Bonus 2: Login Audit

Record:

- Successful login
- Failed login
- Logout
- Password change
- Account lock

Bonus 3: Account Locking

Lock the account after five consecutive failed login attempts.

Bonus 4: Department Hierarchy

Allow departments to have parent and child departments.

Example:

```
Technology
 ├── Software Development
 ├── Infrastructure
 └── Quality Assurance
```

Bonus 5: Employee Document Upload

Allow administrators to upload employee documents such as:

- Offer letter
- Identity document
- Experience certificate

Bonus 6: Notifications

Send an email notification when:

- A transfer request is submitted
- A request is approved
- A request is rejected
- An employee account is disabled

Bonus 7: Docker Deployment

Containerize:

- Spring Boot application
- PostgreSQL
- pgAdmin

Use Docker Compose to run the complete application.

---

18. Final Challenge

Design and implement the system so that authorization is based on all three factors:

```
Role + Resource ownership + Department membership
```

Participants must demonstrate that:

- An administrator can access all resources.
- A manager can access only their department’s resources.
- A normal user can access only their own information.
- Unauthorized data is never exposed in API responses.
- Database updates remain consistent during concurrent operations.